

AiSD Lista 3

Dominik Kowalczyk

16 stycznia 2025

1 Algorytm CUT ROD rozwiązujący problem podziału pręta

Chcemy znaleźć maksymalny zysk po jakim możemy sprzedać pręt długości n , mając możliwość dzielić go na kawałki. W celu rozwiązania tego problemu optymalizacyjnego może pomóc nam algorytm rozcinań pręta - CUT ROD, które przedstawimy implementację naiwną, ze spamiętywaniem oraz iteracyjną.

1.1 Naiwny

Listing 1: Kod funkcji rozcinań pręta w wersji naiwnej

```
1 float CUTROD_NAIWNY(float* p, int n) {  
2     if (n == 0) {  
3         return 0;  
4     }  
5     float q = inf;  
6     for (int i = 1; i <= n; ++i) {  
7         q = max(q, p[i - 1] +  
8             CUTROD_NAIWNY(p, n - i));  
9     }  
10    return q;  
11 }
```

W naszym algorytmie funkcja *CUTROD* przyjmuje argumenty p , który jest wskaźnikiem przechowującym ceny różnych długości pręta oraz n reprezentująca długość pręta.

Na początku wchodzimy do pętli *for* sprawdzająca, czy nasz pręt nie jest długości 0 (jeśli jest zwracamy 0). Następnie tworzy-

my zmienną q przechowującą maksymalny zysk i na start ustawiamy ją na $-\infty$. Następnie przechodzimy do pętli *for* przechodzącej po wszystkich możliwych podziałach pręta. Następnie obliczamy dla każdego podziału maksymalny zysk poprzez wybór za pomocą funkcji *max* zysk aktualnie zapisany pod zmienną q lub ceną odcinka długości i lub pozostałej części pręta $n - i$ poprzez rekurencyjne wywołanie *CUTROD NAIWNY*. Na sam koniec zwracamy q .

1.2 Ze spamiętywaniem

Listing 2: Kod funkcji rozcinania pręta w wersji ze spamiętywaniem

```

1  float MEMORIZED_CUT_ROD(float* p, float* r,
2  int* s, int n) {
3      if (r[n] >= 0) {
4          return r[n];
5      }
6      float q;
7      if (n == 0) {
8          q = 0;
9      } else {
10         q = inf;
11
12         for (int i = 1; i <= n; ++i) {
13
14             float m = p[i - 1] +
15                 MEMORIZED_CUT_ROD(p,
16                 r, s, n - i);
17
18             if (m > q) {
19                 q = m;
20                 s[n] = i;
21             }
22         }
23     }
24     r[n] = q;
25     return q;
26 }
```

W przypadku algorytmu ze spamiętywaniem mamy do czynienia

z większą ilością argumentów. Oprócz p i n mamy wskaźnik (do tablicy) r gdzie przechowywane jest maksymalny możliwy zysk dla danej długości pręta, jak również wskaźnik (do tablicy) s , który przechowuje informacje o optymalnych cięciach pręta.

Mamy warunki początkowe, które działają na tej samej zasadzie co w algorytmie naiwnym. Znacząca zmiana zachodzi w pętli *for* gdzie tworzymy zmienną tymczasową m wyliczając dla każdego cięcia i zysk. Następnie mamy krótką pętlę *if*, która przy warunku, gdy nasz zysk wyliczony i przechowywany w zmiennej tymczasowej m jest większy od q (zysku maksymalnego) to staje się nim, a nasza tablica optymalnego cięcia zapamiętuje wartość i . Ponownie zwracamy nasz zysk maksymalny.

1.3 iteracyjna

Listing 3: Kod funkcji rozcinania pręta w wersji ze spamiętywaniem

```

1  float EXT_BOT_UP_CUT_ROD(float* p, float* r,
2  int* s, int n) {
3      r[0] = 0;
4      for (int j = 1; j <= n; ++j) {
5          float q = inf;
6
7          for (int i = 1; i <= j; ++i) {
8              if (q < p[i - 1] +
9                  r[j - i]) {
10
11                  q = p[i - 1] +
12                      r[j - i];
13
14                  s[j] = i;
15              }
16          }
17          r[j] = q;
18      }
19      return r[n];
20 }
```

W wersji iteracyjnej mamy takie same argumenty jak w przypadku ze spamiętywaniem: p , r , s , n . Początkowo ustawiamy mak-

symalny zysk na 0 ($r[0] = 0$). Wchodzimy do pętli *for*, ustawiamy ponownie q na $-\infty$. Wchodzimy do wewnętrznej pętli *for*, gdzie iterujemy przez wszystkie możliwe cięcia pręta.

1. Dla każdego cięcia obliczana jest suma:

$$\text{zysk} = p[i - 1] + r[j - i],$$

gdzie:

- $p[i - 1]$ - cena odciętego kawałka pręta o długości i
 - $r[j - i]$ - maksymalny zysk dla pozostałej długości pręta $j - i$, (który już wcześniej został obliczony).
2. Jeżeli obliczony zysk jest większy niż dotychczasowa maksymalna wartość q , następuje aktualizacja:
 - q przyjmuje nową, większą wartość zysku,
 - $s[j]$ zapisuje długość cięcia i jako optymalne pierwsze cięcie dla pręta o długości j .

Pętla kończy się, gdy wszystkie możliwe długości cięć zostaną przeanalizowane, a zmienna q zawiera maksymalny możliwy zysk dla pręta o długości j .

2 LCS - problem znajdowania najdłuższego wspólnego podciągu

Przy analizie algorytmu rozwiązującego problem znajdowania najdłuższego wspólnego podciągu przyjrzymy się implementacji rekurencyjnej ze spamiętywaniem oraz wersji iteracyjnej.

2.1 rekurencyjna ze spamiętywaniem

Listing 4: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji rekurencyjnej

```

1 int LCS_R(const string &w1, const string &w2,
2     int m, int n, int** s) {
3     if (s[m][n] != -1) {
4         return s[m][n];
    
```

```

5      }
6      if (m == 0 || n == 0) {
7          return s[m][n] = 0;
8      }
9      if (w1[m - 1] == w2[n - 1]) {
10         return s[m][n] = 1 + LCS_R(w1,
11                                     w2, m - 1, n - 1, s);
12     }
13     return s[m][n] = max(LCS_R(w1, w2, m,
14                                 n - 1, s), LCS_R(w1, w2, m - 1, n, s));
15 }

```

Argumentami funkcji w implementacji rekurencyjnej są jakieś litery oraz ich długości $w1$, $w2$ i odpowiednio m oraz n , a także tablica $s[]$ mająca za zadanie przechowywać maksymalne długości podciągów. Jak w każdej rekurencyjnej funkcji mamy warunki początkowe. Najważniejszą częścią tej implementacji jest podzielenie sytuacji na dwa przypadki - w której mamy styczność z takimi samymi ostatnimi znakami wyrazu (dodajemy 1 i wywołujemy rekurencyjne dla pozostałych) lub są różne. W takim przypadku obliczamy dwie możliwości (z pominięciem ostatniego znaku $w1$ lub $w2$) i zwracamy wartość maksymalną. Tablica $s[]$ jest zwracana i to ona odpowiada za funkcję spamiętywania.

2.2 iteracyjna

Listing 5: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji iteracyjnej

```

1  char** LCS_I(const string &w1, const
2      string &w2, int& dlugosc) {
3      int m = w1.length();
4      int n = w2.length();
5
6      int** c = new int*[m + 1];
7      char** b = new char*[m + 1];
8      for (int i = 0; i <= m; i++) {
9          c[i] = new int[n + 1];
10         b[i] = new char[n + 1];
11     }

```

```

12
13         for (int i = 0; i <= m; i++) {
14             c[i][0] = 0;
15         }
16         for (int j = 0; j <= n; j++) {
17             c[0][j] = 0;
18         }

```

Przypadek iteracyjny został napisany na podstawie pseudokodu, więc nie jest on zbytnio ciekawym kodem. W mojej implementacji mamy tylko trzy argumenty: $w1$, $w2$ oraz zmienna *dlugosc*. Dopiero wewnątrz funkcji definiuje zmienne m oraz n jak również tworzę dwie pomocnicze tablice: $b[]$ odpowiedzialna za przechowywanie kierunków do odtworzenia LCS oraz $c[]$ przechowująca długość ciągu.

Listing 6: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji iteracyjnej

```

1  if (w1[i - 1] == w2[j - 1]) {
2      c[i][j] = c[i - 1][j - 1] + 1;
3      b[i][j] = '\\';
4  } else if (c[i - 1][j] >= c[i][j - 1]) {
5      c[i][j] = c[i - 1][j];
6      b[i][j] = '|';
7  } else {
8      c[i][j] = c[i][j - 1];
9      b[i][j] = '-';
10 }
11 }
12 dlugosc = c[m][n];

```

przeanalizujemy najciekawszy część tego algorytmu:

- Dla każdej pary (i, j) , gdzie $1 \leq i \leq m$ oraz $1 \leq j \leq n$:
 1. Jeśli znaki w obu ciągach są równe, tj. $w1[i-1] = w2[j-1]$, to:

$$c[i][j] = c[i-1][j-1] + 1$$
 oraz ustawiamy kierunek na ukośny
 2. Jeśli znaki są różne, porównujemy wartości:

$$c[i-1][j] \quad \text{oraz} \quad c[i][j-1]$$

– Jeśli $c[i - 1][j] \geq c[i][j - 1]$, to:

$$c[i][j] = c[i - 1][j]$$

oraz ustawiamy kierunek pionowy

– W przeciwnym przypadku:

$$c[i][j] = c[i][j - 1]$$

oraz ustawiamy kierunek poziomy

Na sam koniec przypisujemy do argumentu *dlugosc* wartość najdłuższego wspólnego podciągu. Funkcja zwraca $b[]$.

3 ACTIVITY SELECTOR, czyli problem z przydziałem sal

W przypadku algorytmu rozwiązującego problem przydziału sal skupimy się na wersji rekurencyjnej oraz iteracyjnej wraz z ich modyfikacjami, a także implementacji opartej na programowaniu dynamicznym. Przy czym, pominiemy klasyczne algorytmy wersji rekurencyjnej i iteracyjnej, gdyż były one omawiane na wykładzie i zostały napisane wprost na podstawie pseudokodu.

3.1 Zmodyfikowana rekurencja oraz iteracja

Modyfikacja zarówno wersji rekurencyjnej jak i iteracyjnej polega na tym samym, czyli by działał na danych wejściowych posortowanych względem czasu rozpoczęcia. Przyjmujemy następujące argumenty:

- $s[]$ - tablica przechowująca czasy rozpoczęcia zajęć.
- $f[]$ - tablica przechowująca czasy zakończenia zajęć.
- k - indeks aktualnie rozważanego zajęcia.
- $zaj[]$ - tablica, w której zapisywane są indeksy wybranych zajęć.
- $ilosc$ - referencja do zmiennej liczącej liczbę wybranych zajęć.
- n - długość tablicy zajęcia.

Listing 7: Kod funkcji rozwiązywania problemu przydziału sal w wersji zmodyfikowanej rekurencji

```

1 void REC_ACTIVITY_SELECTOR_MOD( float s [ ] ,
2   float f [ ] , int k, int zaj [ ] , int &ilosc ) {
3   int m = k - 1;
4
5   while (m > 0 && f [m] > s [k]) {
6     m--;
7   }
8   if (m > 0) {
9     zaj [ ilosc++ ] = m;
10    REC_ACTIVITY_SELECTOR_MOD(s , f ,
11      m, zaj , ilosc );
12  }
13 }
```

zaczynamy nasze rozważania od zajęć następujących po aktualnych. Pętla *while* poszukuje pierwszego zajęcia (*m*), które kończy się przed rozpoczęciem aktualnego zajęcia *k*. Jeśli warunek jest spełniony to zmniejszamy *m* aby przejść do wcześniejszego zajęcia. Jeżeli udało się takowe znaleźć to indeks *m* jest dodawany do tablicy *zaj*[]. Wywołujemy rekurencję.

Listing 8: Kod funkcji rozwiązywania problemu przydziału sal w wersji zmodyfikowanej iteracji

```

1 void ACTIVITY_SELECTOR_MOD( float s [ ] ,
2   float f [ ] , int n, int zaj [ ] , int &ilosc ) {
3   zaj [ ilosc++ ] = n - 1;
4   int k = n - 1;
5
6   for (int m = n - 2; m >= 1; m--) {
7     if ( f [m] <= s [k] ) {
8       zaj [ ilosc++ ] = m;
9       k = m;
10    }
11  }
12 }
```

W przypadku iteracji ponownie wybieramy ostatnie zajęcie, jak to od którego, zaczynamy i idziemy w dół. Sprawdzamy, czy zajęcia o indeksie *m* (które kończy się w czasie *f*[*m*]) kończy się przed

rozpoczęciem ostatnio wybranego zajęcia ($s[k]$). Jeśli warunek jest spełniony, to możemy wybrać to zajęcie, ponieważ nie nakłada się na ostatnio wybrane i dodajemy je do tablicy $zaj[]$.

3.2 Dynamiczna

Listing 9: Kod funkcji rozwiązywania problemu przydziału sal w wersji opartej o programowanie dynamiczne

```

1 void ACT_SEL_D(float s[], float f[], int n) {
2     int **c = new int *[n + 2];
3     int **b = new int *[n + 2];
4     for (int i = 0; i < n + 2; ++i) {
5         c[i] = new int[n + 2]();
6         b[i] = new int[n + 2]();
7     }

```

Schemat działania oraz implementacji tego kodu jest bardzo podobny do iteracyjnej wersji LCS, gdzie tworzyliśmy tablicę, które pomagały nam dzielić problem na podproblemy. Tutaj mamy podobnie z tym, że długość tablic jest długości $n + 2$, gdyż przy algorytmie ACTIVITY_SELECTOR musimy pamiętać o fejkowych zmiennych (takich które potrzebują miejsca, ale nie są brane do wyniku ostatecznego).

Listing 10: Kod funkcji rozwiązywania problemu przydziału sal w wersji opartej o programowanie dynamiczne

```

1 for (int k = i + 1; k < j; ++k) {
2     if (f[i] <= s[k] && f[k] <= s[j]) {
3         int elem = c[i][k] + c[k][j] + 1;
4         if (elem > c[i][j]) {
5             c[i][j] = elem;
6             b[i][j] = k;
7         }
8     }
9 }

```

Przyjrzyjmy się najbardziej skomplikowanej pętli w tym kodzie:

1. `for (int k = i + 1; k < j; ++k)`

Pętla iteruje przez wszystkie możliwe wartości indeksu k w przedziale od $i+1$ do $j-1$. Celem jest znalezienie odpowiedniego zajęcia k ,

które może dzielić przedział $[i, j]$ na dwa mniejsze, niezależne pod-problemy.

2. Warunek 2 sprawdza, czy aktywność k nie nakłada się na inne w przedziale $[i, j]$.

3. `int element = c[i][k] + c[k][j] + 1;`

Obliczamy wartość `element`, która reprezentuje liczbę zajęć, które można wybrać w przedziale $[i, j]$. `c[i][k]` to maksymalna liczba aktywności w przedziale $[i, k]$, `c[k][j]` to maksymalna liczba aktywności w przedziale $[k, j]$, a `+ 1` uwzględnia aktywność k .

4. `if (element > c[i][j])`

Sprawdzamy, czy liczba zajęć obliczona dla podziału przez k jest większa niż obecna wartość w `c[i][j]`.

5. `c[i][j] = element;`

Jeśli nowa liczba aktywności jest większa, to zapisujemy ją w `c[i][j]`, aktualizując maksymalną liczbę zajęć dla przedziału $[i, j]$.

6. `b[i][j] = k;`

Zapisujemy indeks k w tablicy `b[i][j]`, ponieważ k jest najlepszym punktem podziału przedziału $[i, j]$ na dwa podproblemy.

4 Porównania

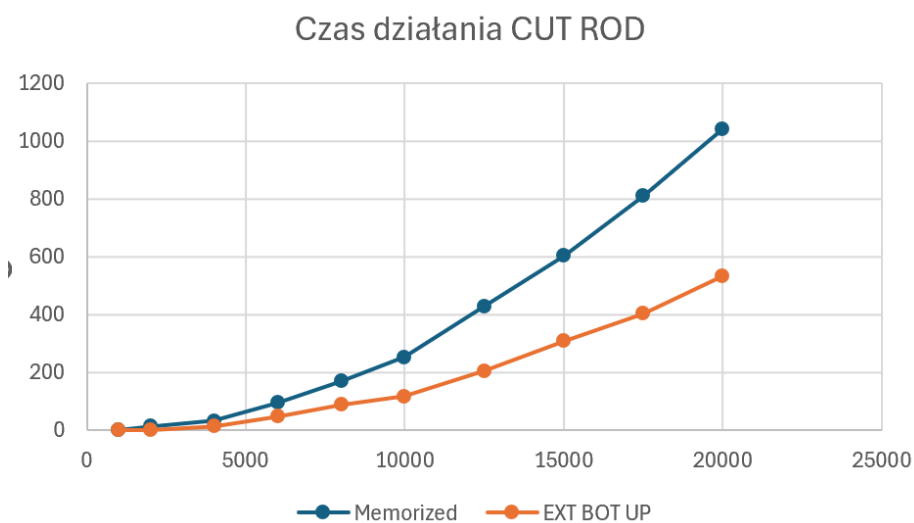
Podsumowując, dokonaliśmy testów trzech różnych algorytmów oraz ich najróżniejszej implementacji. W CUT RODZIE najbardziej optymalną wersją był EXT BOT UP, a najgorszą wersja NAIWNA. W przypadku LCS zwyciężył sposób iteracyjny, który na większych długościach n spisywał się znacznie lepiej do wersji rekurencyjnej. Na koniec algorytm ACTIVITY SELECTOR, gdzie modyfikacje nie okazały się być wydajniejsze od klasycznych implementacji. Nie mówiąc już nic, o tragicznie wypadającą implementacją opartą o programowanie dynamiczne. Najlepszym okazał się klasyczny iteracyjny, który nieznacznie lepiej wypada w testach efektywności pracy.

CUT ROD NAIWNY	
długość n ▼	Czas (ms) ▼
5	0
10	0
15	2
20	7
25	241
30	7269

Rysunek 1: Tabela czasu działania algorytmu CUT ROD w wersji naiwnej.

CUT ROD - czas działania (ms)			
długość n ▼	Memorized ▼	EXT BOT UP ▼	
1000	0	0	
2000	14	0	
4000	33	15	
6000	96	47	
8000	170	89	
10000	253	117	
12500	429	206	
15000	604	308	
17500	810	403	
20000	1041	532	

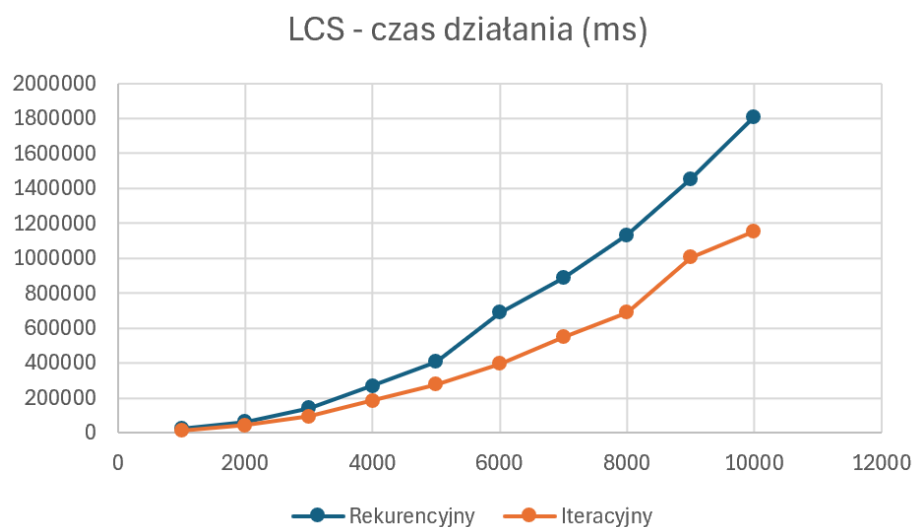
Rysunek 2: Tabela czasu działania algorytmu CUT ROD w wersji ze spamiętywaniem oraz iteracyjną.



Rysunek 3: Wykres czasu działania algorytmu CUT ROD w wersji ze spamiętywaniem oraz iteracyjną.

LCS - czas działania (ms)			
długość n	Rekurencyjny	Iteracyjny	
1000	21850	10933	
2000	62968	42695	
3000	141696	94131	
4000	268662	183661	
5000	406291	278134	
6000	689091	394154	
7000	886726	547502	
8000	1132544	688956	
9000	1454488	1004347	
10000	1805217	1150444	

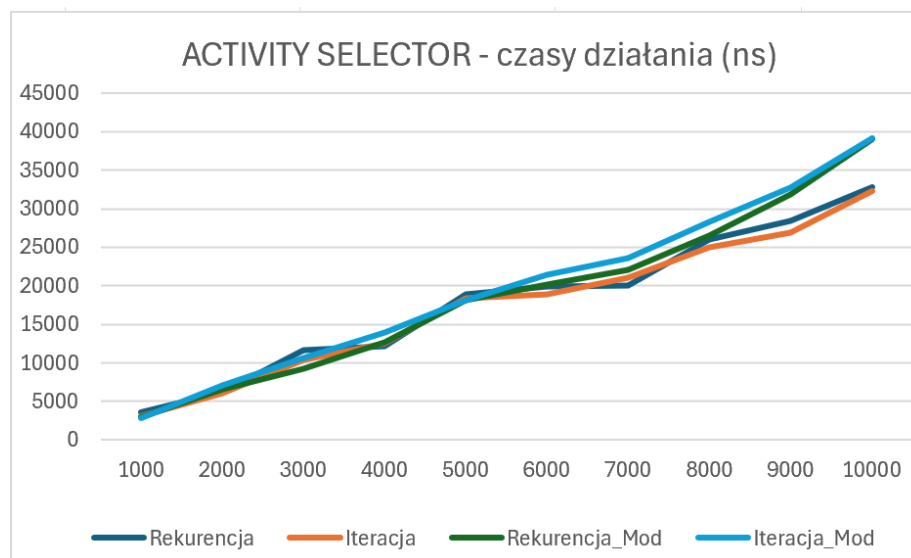
Rysunek 4: Wykres czasu działania algorytmu LCS w wersji rekurencyjnej ze spamiętywaniem oraz iteracyjnej.



Rysunek 5: Wykres czasu działania algorytmu CUT ROD w wersji naiwnej.

ACTIVITY SELECTOR - czasy działania (ns)						
dlugość n	Rekurencja	Iteracja	Rekurencja_Mod	Iteracja_Mod	Dynamiczna	
1000	3556	3057	2921	2836	1479685600	
2000	6164	6010	6540	7140	15516424100	
3000	11629	10342	9195	10613	68182831300	
4000	12168	12540	12627	13977	1.82683E+11	
5000	18875	18409	18146	18164		
6000	19898	18909	20107	21455		
7000	20005	21024	22059	23546		
8000	26061	25021	26478	28246		
9000	28442	26941	31849	32712		
10000	32790	32247	38966	39207		

Rysunek 6: Tabela czasu działania algorytmu ACTIVITY SELECTOR.



Rysunek 7: Wykres czasu działania algorytmu ACTIVITY SELECTOR.